

Pages Directory > ... > Deploying > Static Exports

Static Exports

Note: Support for static exports was added in Next.js 13.3.

Next.js enables starting as a static site or Single-Page Application (SPA), then later optionally upgrading to use features that require a server.

The core of Next.js has been designed to enable starting as a static site (or Single-Page Application), if desired, and later upgrade to use powerful, dynamic features that require a server. For example, [Incremental Static Regeneration](#), [Internationalized Routing](#), and [more](#).

Since Next.js supports this static export, it can be deployed and hosted on any web server that can serve HTML/CSS/JS static assets.

Configuration

To enable a static export, change the output mode inside `next.config.js`:

```
JS next.config.js

1  /**
2   * @type {import('next').NextConfig}
3   */
4  const nextConfig = {
5    output: 'export',
6  };
7
```

```
8 module.exports = nextConfig;
```

After running `next build`, Next.js will produce an `out` folder which contains the HTML/CSS/JS assets for your application.

You can utilize `getStaticProps` and `getStaticPaths` to generate an HTML file for each page in your `pages` directory (or more for [dynamic routes](#)).

If you want to change the output directory, you can configure `distDir` like the following:

```
1 /**
2  * @type {import('next').NextConfig}
3  */
4  const nextConfig = {
5    output: 'export',
6    distDir: 'dist',
7  };
8
9  module.exports = nextConfig;
```

In this example, `next build` will generate a `dist` directory containing the HTML/CSS/JS static assets.

Learn more about [Setting a custom build directory](#).

If you want to change the output directory structure to always include a trailing slash, you can configure `trailingSlash` like the following:

```
1 /**
2  * @type {import('next').NextConfig}
3  */
4  const nextConfig = {
5    output: 'export',
6    trailingSlash: true,
7  };
8
9  module.exports = nextConfig;
```

This will change links so that `href="/about"` will instead be `href="/about/"`. It will also change the output so that `out/about.html` will instead emit `out/about/index.html`.

Learn more about [Trailing Slash](#).

Supported Features

The core of Next.js has been designed to support static exports.

Supported Features

The majority of core Next.js features needed to build a static site are supported, including:

- [Dynamic Routes when using `getStaticPaths`](#)
- Prefetching with `next/link`
- Preloading JavaScript
- [Dynamic Imports](#)
- Any styling options (e.g. CSS Modules, styled-jsx)
- [Client-side data fetching](#)
- `getStaticProps`
- `getStaticPaths`
- [Image Optimization](#) using a [custom loader](#)

`getInitialProps`

It's possible to use the `getInitialProps` API instead of `getStaticProps`, but it comes with a few caveats:

- `getInitialProps` cannot be used alongside `getStaticProps` or `getStaticPaths` on any given page. If you have dynamic routes, instead of using `getStaticPaths` you'll need to configure the `exportPathMap` parameter in your `next.config.js` file to let the exporter know which HTML files it should output.
- When `getInitialProps` is called during export, the `req` and `res` fields of its `context` parameter will be empty objects, since during export there is no server running.
- `getInitialProps` **will be called on every client-side navigation**, if you'd like to only fetch data at build-time, switch to `getStaticProps`.
- `getInitialProps` should fetch from an API and cannot use Node.js-specific libraries or the file system like `getStaticProps` can.

We recommend migrating towards `getStaticProps` over `getInitialProps` whenever possible.

Image Optimization

[Image Optimization](#) through `next/image` can be used with a static export by defining a custom image loader in `next.config.js`. For example, you can optimize images with a service like Cloudinary:

```
JS next.config.js

1  /** @type {import('next').NextConfig} */
2  const nextConfig = {
3    output: 'export',
4    images: {
5      loader: 'custom',
6      loaderFile: './app/image.ts',
7    },
8    experimental: {
9      appDir: true,
10   },
11 };
12
13 module.exports = nextConfig;
```

This custom loader will define how to fetch images from a remote source. For example, the following loader will construct the URL for Cloudinary:

TS app/image.ts



```
1 export default function cloudinaryLoader({
2   src,
3   width,
4   quality,
5 }: {
6   src: string,
7   width: number,
8   quality?: number,
9 }) {
10   const params = ['f_auto', 'c_limit', `w_${width}`, `q_${quality || 'auto'}`];
11   return `https://res.cloudinary.com/demo/image/upload/${params.join(
12     ',',
13   )}${src}`;
14 }
```

You can then use `next/image` in your application, defining relative paths to the image in Cloudinary:

TS app/page.tsx



```
1 import Image from 'next/image';
2
3 export default function Page() {
4   return <Image alt="turtles" src="/turtles.jpg" width={300} height={300} />;
5 }
```

Route Handlers

Route Handlers will render a static response when running `next build`. Only the `GET` HTTP verb is supported. This can be used to generate static HTML, JSON, TXT, or other files from dynamic or static data. For example:

TS app/data.json/route.ts



```
1 import { NextResponse } from 'next/server';
2
3 export async function GET() {
4   return NextResponse.json({ name: 'Lee' });
5 }
```

```
5 }
```

The above file `app/data.json/route.ts` will render to a static file during `next build`, producing `data.json` containing `{ name: 'Lee' }`.

If you need to read dynamic values from the incoming request, you cannot use a static export.

Browser APIs

Client Components are pre-rendered to HTML during `next build`. Because [Web APIs](#) like `window`, `localStorage`, and `navigator` are not available on the server, you need to safely access these APIs only when running in the browser. For example:

```
1 'use client';
2
3 import { useEffect } from 'react';
4
5 export default function ClientComponent() {
6   useEffect(() => {
7     // You now have access to `window`
8     console.log(window.innerHeight);
9   }, [])
10
11   return ...;
12 }
```

Unsupported Features

Features that require a Node.js server, or dynamic logic that cannot be computed during the build process, are not supported:

- [Image Optimization](#) (default loader)
- [Internationalized Routing](#)
- [API Routes](#)
- [Rewrites](#)

- [Redirects](#)
- [Headers](#)
- [Middleware](#)
- [Incremental Static Regeneration](#)
- `getStaticPaths` with `fallback: true`
- `getStaticPaths` with `fallback: 'blocking'`

`getServerSideProps`

- [Image Optimization](#) (default loader)
- [Internationalized Routing](#)
- [API Routes](#)
- [Rewrites](#)
- [Redirects](#)
- [Headers](#)
- [Middleware](#)
- [Incremental Static Regeneration](#)
- `getStaticPaths` with `fallback: true`
- `getStaticPaths` with `fallback: 'blocking'`
- `getServerSideProps`



> Menu

With a static export, Next.js can be deployed and hosted on any web server that can serve HTML/CSS/JS static assets.

When running `next build`, Next.js generates the static export into the `out` folder. Using `next export` is no longer needed. For example, let's say you have the following routes:

- `/`
- `/blog/[id]`

After running `next build`, Next.js will generate the following files:

- `/out/index.html`
- `/out/404.html`
- `/out/blog/post-1.html`
- `/out/blog/post-2.html`

If you are using a static host like Nginx, you can configure rewrites from incoming requests to the correct files:

nginx.conf



```
1  server {
2      listen 80;
3      server_name acme.com;
4
5      root /var/www;
6
7      location / {
8          try_files /out/index.html =404;
9      }
10
11     location /blog/ {
12         rewrite ^/blog/(.*)$ /out/blog/$1.html break;
13     }
14
15     error_page 404 /out/404.html;
16     location = /404.html {
17         internal;
18     }
19 }
```


next export (deprecated)

Warning: "next export" is deprecated since Next.js 13.3 in favor of "output: 'export'" configuration.

In versions of Next.js prior to 13.3, there was no configuration option in `next.config.js` and instead there was a separate command for `next export`.

This could be used by updating your `package.json` file to include `next export` like the following:

```
1  "scripts": {  
2    "build": "next build && next export"  
3  }
```

Running `npm run build` will generate an `out` directory.

`next export` builds an HTML version of your app. During `next build`, `getStaticProps` and `getStaticPaths` will generate an HTML file for each page in your `pages` directory (or more for `dynamic routes`). Then, `next export` will copy the already exported files into the correct directory. `getInitialProps` will generate the HTML files during `next export` instead of `next build`.

Warning: Using `exportPathMap` is deprecated and is overridden by `getStaticPaths` inside `pages`. We recommend not to use them together.

General Resources

[Docs](#)[Learn](#)[Showcase](#)[Blog](#)[Analytics](#)[Next.js Conf](#)[Previews](#)